

Basics

```

//////////////////// BASICS //////////////////////

#define ld long double
const ld M_PI2 = 3.14159265358979323846
const ld EPS = 1e-8;

// complex to point
typedef long double T;
typedef complex<T> pt;
#define x real()
#define y imag()

bool operator==(pt a, pt b) {return fabs(a.x - b.x) < EPS && fabs(a.y - b.y) <
EPS;}
bool operator!=(pt a, pt b) {return !(a == b);}

int sgn(T val) {
    return (T(0) < val) - (val < T(0));
}

T sq(pt p) {return p.x*p.x + p.y*p.y;}

T dot(pt v, pt w) {return v.x*w.x + v.y*w.y;}
T cross(pt v, pt w) {return v.x*w.y - v.y*w.x;}

void builtin_functions_demo(){
    pt p;

    // Returns angle (in radians) from origin to point p (range: (- $\pi$ ,  $\pi$ ])
    cout << "Angle (degrees): " << arg(p) * 180 / M_PI << '\n';

    // Euclidean distance from origin (magnitude)
    cout << "Distance (magnitude): " << abs(p) << '\n';

    ld angle_rad = arg(p);
    ld angle_deg = angle_rad * 180 / M_PI;

    cout << fixed << setprecision(10);
    cout << "sin(angle): " << sin(angle_rad) << '\n';
    cout << "cos(angle): " << cos(angle_rad) << '\n';
    cout << "tan(angle): " << tan(angle_rad) << '\n';

    // Inverse trig functions:
    // acos:  $\cos^{-1}(x)$ , returns angle in radians in  $[0, \pi]$ 
    // asin:  $\sin^{-1}(x)$ , returns angle in radians in  $[-\pi/2, \pi/2]$ 
    // atan:  $\tan^{-1}(x)$ , returns angle in radians in  $[-\pi/2, \pi/2]$ 
    // atan2(y, x): returns angle from (0, 0) to (x, y), full circle  $(-\pi, \pi]$ 

```

```
ld value = 0.5; // example value between -1 and 1

cout << "acos(0.5): " << acos(value) * 180 / M_PI << " degrees\n";
cout << "asin(0.5): " << asin(value) * 180 / M_PI << " degrees\n";
cout << "atan(1.0): " << atan(1.0) * 180 / M_PI << " degrees\n";

// Example: angle from origin to point (3, 4)
cout << "atan2(4, 3): " << atan2(4, 3) * 180 / M_PI << " degrees\n";
}
```

 alt text

Transfromations

```
//////////////////// TRANSFORMATIONS //////////////////////

// Translate point p by vector v
pt translate(pt v, pt p) {
    return p + v;
}

// Scale point p with respect to center c by a scalar factor
// - factor > 1 => expansion
// - factor < 1 => contraction
// - factor < 0 => includes reflection
pt scale(pt c, ld factor, pt p) {
    return c + (p - c) * factor;
}

// Rotate point p around center c by angle (in radians, counter-clockwise)
// - Uses polar form multiplication
// - Positive angle: counter-clockwise
// - Negative angle: clockwise
pt rot(pt c, ld angle, pt p) {
    return c + (p - c) * polar(1.0l, angle);
}

// Apply linear transformation that maps:
// - point p → fp
// - point q → fq
// Then computes the corresponding image of point r
// Assumes q ≠ p, and the transformation is linear
pt linearTransfo(pt p, pt q, pt r, pt fp, pt fq) {
    return fp + (r - p) * (fq - fp) / (q - p);
}

// Reflect point p through center c (central symmetry)
pt reflect_point(pt c, pt p) {
    return c * 2.0l - p;
}
```

Angles

```

//////////////////// ANGLES //////////////////////

// Normalize angle to range  $[-\pi, \pi)$ 
ld normalize_angle(ld angle) {
    while (angle <= -M_PI) angle += 2 * M_PI;
    while (angle > M_PI) angle -= 2 * M_PI;
    return angle;
}

T to_deg(T rad) { return rad * 180 / M_PI; }
T to_rad(T deg) { return deg * M_PI / 180; }

// Returns the unsigned angle between two vectors p and v
// Range:  $[0, \pi]$ 
// Uses  $\text{acos}(\text{dot} / (|p||v|))$ , clamped for precision safety
T angle(pt p, pt v) {
    return acos(clamp(dot(p, v) / abs(p) / abs(v), (T)-1.0, (T)1.0));
}

// Checks if two vectors are perpendicular (dot product  $\sim 0$ )
bool isPerp(pt p, pt v) {
    return fabs(dot(p, v)) < EPS;
}

// Returns the left perpendicular vector to v (rotated  $90^\circ$  counter-clockwise)
// For  $v = (a, b)$ , returns  $(-b, a)$ 
pt perp(pt v) {
    return {-y(v), x(v)};
}

// Orientation test:
// - Returns positive if c is to the left of line  $a \rightarrow b$  (counter-clockwise turn)
// - Returns negative if to the right (clockwise turn)
// - Zero if collinear
T orient(pt a, pt b, pt c) {
    return cross(b - a, c - a);
}

// Computes the directed angle  $\angle BAC$  in  $[0, 2\pi)$ 
// - If c is to the left of ab (counter-clockwise turn), returns angle as is
// - If to the right, returns  $2\pi - \text{angle}$ 
T oriented_angle(pt a, pt b, pt c) {
    T ampli = angle(b - a, c - a);
    if (orient(a, b, c) < 0) ampli = 2 * M_PI - ampli;
    return ampli;
}

```

```

// Like oriented_angle but returns signed angle in  $(-\pi, \pi]$ 
// - Positive = counter-clockwise turn
// - Negative = clockwise turn
T traveled_angle(pt a, pt b, pt c) {
    T ampli = angle(b - a, c - a);
    if (orient(a, b, c) < 0) ampli = -ampli;
    return ampli;
}

// Checks if point p lies within angle  $\angle BAC$  (excluding reflex part)
// - Works for both convex and reflex angles
// - Direction is counter-clockwise from b to c
// - Returns true if p lies in the "swept" region from b to c around a
bool inAngle(pt a, pt b, pt c, pt p){
    T abp = orient(a, b, p), acp = orient(a, c, p), abc = orient(a, b, c);
    if(abc < 0) swap(abp, acp);
    return (abp >= 0 && acp <= 0) ^ (abc < 0);
}

// Returns the angle bisector of vectors a and b (from origin)
// Not normalized
pt angle_bisector(pt a, pt b) {
    return a / abs(a) + b / abs(b);
}

```

Lines

```

//////////////////// LINES //////////////////////

// Line represented as:  $v \cdot p = c \iff \text{cross}(v, p) = c$ 
// v is a direction vector (not necessarily normalized)
// c is the offset such that the line passes through point(s) satisfying the
equation
struct line {
    pt v; // direction vector of the line
    T c; // constant such that  $\text{cross}(v, p) == c$  for points p on the line

    // Construct from direction vector and offset
    line(pt v, T c) : v(v), c(c) {}

    // Construct from general equation:  $ax + by = c$ 
    // Equivalent to  $\text{cross}((b, -a), (x, y)) = c$ 
    line(T a, T b, T _c) {
        v = {b, -a};
        c = _c;
    }

    // Construct line passing through points p and q
    // Direction is (q - p), offset is  $\text{cross}(v, p)$ 
    line(pt p, pt q) {
        v = q - p;
        c = cross(v, p);
    }

    // Returns signed distance from point p to the line
    // Positive: p is to the left of the line (if v points forward)
    T side(pt p) { return cross(v, p) - c; }

    // Euclidean distance from point to line
    double dist(pt p) { return abs(side(p)) / abs(v); }

    // Squared distance from point to line (avoids sqrt)
    double sqDist(pt p) { return side(p) * side(p) / (T)sq(v); }

    // Returns perpendicular line to this line that passes through point p
    line perpThrough(pt p) { return {p, p + perp(v)}; }

    // Compare projections of p and q onto the line direction (for ordering)
    bool cmpProj(pt p, pt q) { return dot(v, p) < dot(v, q); }

    // Returns translated version of the line by vector t
    line translate(pt t) { return {v, c + cross(v, t)}; }

    // -----
    // Floating-point only utilities:

```

```

// -----

// Shift line to the left by a given distance (based on direction of v)
line shiftLeft(double dist) { return {v, c + dist * abs(v)}; }

// Projects point p onto the line (closest point on line)
pt proj(pt p) { return p - perp(v) * side(p) / sq(v); }

// Reflects point p across the line (mirror image)
pt refl(pt p) { return p - perp(v) * (T)2.0 * side(p) / sq(v); }
};

// ===== LINE OPERATIONS ===== //

// Finds intersection point between two lines (if exists)
// Returns false if lines are parallel (no intersection)
// Otherwise, stores intersection point in `out`
bool inter(line l1, line l2, pt &out) {
    T d = cross(l1.v, l2.v);
    if (fabs(d) < EPS) return false; // Parallel lines
    out = (l2.v * l1.c - l1.v * l2.c) / d;
    return true;
}

// Returns angle bisector line between l1 and l2
// - If `interior` is true → internal bisector
// - If `interior` is false → external bisector
// Assumes lines are not parallel
line bisector(line l1, line l2, bool interior) {
    assert(cross(l1.v, l2.v) != 0); // Must not be parallel
    ld sign = interior ? 1.0 : -1.0;
    return {
        l2.v / abs(l2.v) + l1.v / abs(l1.v) * sign,
        l2.c / abs(l2.v) + l1.c / abs(l1.v) * sign
    };
}

```

Segments

```

//////////////////// SEGMENTS //////////////////////

// Checks whether point p lies inside the disk with diameter endpoints a and b
// (i.e., p lies between a and b in terms of projection)
bool inDisk(pt a, pt b, pt p) {
    return dot(a - p, b - p) <= EPS;
}

// Returns true if point p lies on segment ab (inclusive)
bool onSegment(pt a, pt b, pt p) {
    return abs(orient(a, b, p)) <= EPS && inDisk(a, b, p);
}

// Checks if segments ab and cd properly intersect (i.e., non-collinear, interior
// cross)
// If they do, stores intersection point in `out` and returns true
bool properInter(pt a, pt b, pt c, pt d, pt &out) {
    T oa = orient(c, d, a);
    T ob = orient(c, d, b);
    T oc = orient(a, b, c);
    T od = orient(a, b, d);
    // Proper intersection ⇔ opposite orientations on both sides
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0) {
        out = (a * ob - b * oa) / (ob - oa);
        return true;
    }
    return false;
}

// Returns set of all intersection points between segments ab and cd
// May include:
// - No intersection (empty)
// - One point (touching or proper intersect)
// - Multiple points (segment overlap)
// Points are returned as pairs of coordinates
set<pair<ld, ld>> inters(pt a, pt b, pt c, pt d) {
    set<pair<ld, ld>> s;
    pt out;

    // Endpoints that coincide
    if (a == c || a == d) s.insert({x(a), y(a)});
    if (b == c || b == d) s.insert({x(b), y(b)});
    if (!s.empty()) return s;

    // Proper intersection
    if (properInter(a, b, c, d, out)) {
        return { {x(out), y(out)} };
    }
}

```



```
// Collinear overlaps
if (onSegment(c, d, a)) s.insert({x(a), y(a)});
if (onSegment(c, d, b)) s.insert({x(b), y(b)});
if (onSegment(a, b, c)) s.insert({x(c), y(c)});
if (onSegment(a, b, d)) s.insert({x(d), y(d)});

return s;
}

// Returns the shortest distance from point p to segment ab
ld segPoint(pt a, pt b, pt p) {
    if (a != b) {
        line l(a, b);
        if (l.cmpProj(a, p) && l.cmpProj(p, b)) {
            return l.dist(p); // closest point is inside the segment
        }
    }
    // Otherwise, closest to endpoint
    return min(abs(p - a), abs(p - b));
}

// Returns the shortest distance between two segments ab and cd
// If they intersect, returns 0
ld segSeg(pt a, pt b, pt c, pt d) {
    pt dummy;
    if (properInter(a, b, c, d, dummy))
        return 0;
    return min({
        segPoint(a, b, c),
        segPoint(a, b, d),
        segPoint(c, d, a),
        segPoint(c, d, b)
    });
}
```

Polygons

```

//////////////////// POLYGONS //////////////////////

// Area of triangle ABC using cross product
ld areaTriangle(pt a, pt b, pt c) {
    return fabs(cross(b - a, c - a)) / (T)2.0;
}

// Signed area of a polygon (positive if CCW, negative if CW)
// Points must form a simple polygon (no self-intersections)
ld areaPolygon(vector<pt> p) {
    ld area = 0;
    for (int i = 0, n = p.size(); i < n; i++) {
        area += cross(p[i], p[(i + 1) % n]);
    }
    return area / 2.0;
}

// Helper: returns true if point p is above or on same height as a
bool above(pt a, pt p) {
    return p.y >= a.y;
}

// Returns true if segment [p, q] crosses the horizontal ray to the right of a
bool crossesRay(pt a, pt p, pt q) {
    return (above(a, q) - above(a, p)) * orient(a, p, q) > 0;
}

// Determines if point `a` is inside polygon `p`
// strict = true → point on edge returns false
// strict = false → point on edge returns true
bool inPolygon(vector<pt> p, pt a, bool strict = true) {
    int numCrossings = 0;
    for (int i = 0, n = p.size(); i < n; i++) {
        pt curr = p[i], next = p[(i + 1) % n];
        if (onSegment(curr, next, a))
            return !strict;
        numCrossings += crossesRay(a, curr, next);
    }
    return numCrossings & 1; // odd = inside, even = outside
}

```

Circles

```

//////////////////// CIRCLES //////////////////////

// Returns circumcircle (center, radius) of triangle ABC
// The result is a pair<pt, T> where pt is the center and T is the radius
pair<pt, T> circumCircle(pt a, pt b, pt c) {
    b = b - a;
    c = c - a;
    assert(cross(b, c) != 0); // No circle if points are collinear
    pt center = a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / (T)2;
    T radius = abs(center - a); // Or abs(...) since center = a + ...
    return {center, radius};
}

// Intersects a circle with center o and radius r with line l
// If they intersect, fills out with two points (equal if tangent)
// Returns 0 = no intersection, 1 = tangent, 2 = secant
int circleLine(pt o, double r, line l, pair<pt, pt>& out) {
    double h2 = r * r - l.sqDist(o); // squared distance from circle to line
    if (h2 >= 0) {
        pt p = l.proj(o); // closest point on line
        pt h = l.v * (T)(sqrt(h2) / abs(l.v)); // scaled direction vector
        out = {p - h, p + h};
    }
    return 1 + sgn(h2);
}

// Intersects two circles at most at two points
// Returns 0 = no intersection, 1 = tangent, 2 = intersecting
int circleCircle(pt o1, T r1, pt o2, T r2, pair<pt, pt>& out) {
    pt d = o2 - o1;
    T d2 = sq(d);
    if (d2 == 0) { assert(r1 != r2); return 0; } // concentric
    T pd = (d2 + r1*r1 - r2*r2) / 2;
    T h2 = r1*r1 - pd*pd / d2;
    if (h2 >= 0) {
        pt p = o1 + d * pd / d2;
        pt h = perp(d) * sqrt(h2 / d2);
        out = {p - h, p + h};
    }
    return 1 + sgn(h2);
}

// Finds the common tangents between two circles
// inner = true → internal tangents, false → external
// Returns 0 (no tangent), 1 (one tangent), or 2 (two tangents)
int tangents(pt o1, T r1, pt o2, T r2, bool inner, vector<pair<pt, pt>>& out) {
    if (inner) r2 = -r2;
    pt d = o2 - o1;

```

```

    T dr = r1 - r2, d2 = sq(d), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) { assert(h2 != 0); return 0; } // concentric or nested
    for (T sign : {-1, 1}) {
        pt v = (d * dr + perp(d) * sqrt(h2) * sign) / d2;
        out.push_back({o1 + v * r1, o2 + v * r2});
    }
    return 1 + (h2 > 0);
}

T circleCircleAreaIntersection(pt o1, T r1, pt o2, T r2) {
    T d = abs(o1 - o2); // distance between centers

    // No overlap
    if (d >= r1 + r2) return 0;

    // One completely inside the other
    if (d <= abs(r1 - r2)) {
        T r = min(r1, r2);
        return M_PI * r * r;
    }

    T a1 = r1*r1, a2 = r2*r2;

    T alpha = 2 * acos(clamp((a1 + d*d - a2) / (2 * r1 * d), (T)-1, (T)1));
    T beta  = 2 * acos(clamp((a2 + d*d - a1) / (2 * r2 * d), (T)-1, (T)1));

    T area1 = 0.5 * a1 * (alpha - sin(alpha));
    T area2 = 0.5 * a2 * (beta - sin(beta));

    return area1 + area2;
}

```

Pick's Theorem

```

typedef int T;
typedef complex<T> pt;
#define x real()
#define y imag()

T cross(pt v, pt w) { return v.x * w.y - v.y * w.x; }

// Returns 2 * area of polygon (always non-negative)
int areaPolygon(const vector<pt>& p) {
    int area = 0;
    for (int i = 0, n = p.size(); i < n; i++)
        area += cross(p[i], p[(i+1)%n]);
    return abs(area);
}

// Counts lattice points on the segment from A to B (inclusive)
int latticePointsOnSegment(pt A, pt B) {
    return __gcd(abs(A.x - B.x), abs(A.y - B.y));
}

// Computes total boundary lattice points of polygon
int calcBoundary(const vector<pt>& p) {
    int n = p.size(), total = 0;
    for (int i = 0; i < n; ++i)
        total += latticePointsOnSegment(p[i], p[(i+1)%n]);
    return total;
}

void takePoint(pt& p){
    int x, y; cin >> x >> y;
    p = {x, y};
}

void solve() {
    int n; cin >> n;
    vector<pt> p(n);
    for (int i = 0; i < n; ++i) takePoint(p[i]);

    int b = calcBoundary(p);
    int a = areaPolygon(p);
    int i = (a - b + 2) / 2;

    cout << i << " " << b << '\n';
}

```



Line sweeping

```
void solve() {
    int n; cin >> n;
    vector<pt> p(n);
    for (int i = 0; i < n; ++i) takePoint(p[i]);

    sort(all(p), [&](pt & a, pt & b)->bool{
        return a.x < b.x;
    });

    set<array<int, 2>> window;

    int ans = 8e18 + 5, j = 0, bestSq = sqrt(ans);

    for (int i = 0; i < n; ++i) {
        while (j < n && p[i].x - p[j].x > bestSq) {
            window.erase({p[j].y, p[j].x});
            j++;
        }

        auto st = window.lower_bound({p[i].y - bestSq, -oo});
        auto en = window.lower_bound({p[i].y + bestSq, -oo});

        if(en != window.end()) en++;
        while (st != en){
            pt cur = {(*st)[1], (*st)[0]};
            ans = min(ans, sq(p[i] - cur));
            bestSq = min(bestSq, abs(p[i] - cur));
            st++;
        }

        window.insert({p[i].y, p[i].x});
    }

    cout << ans << endl;
}
```

Extra

```

ld perimeterPolygon(const vector<pt> &p) {
    ld peri = 0;
    int n = p.size();
    for (int i = 0; i < n; i++)
        peri += abs(p[i] - p[(i + 1) % n]);
    return peri;
}

pt centroidPolygon(const vector<pt> &p) {
    ld A = 0;
    pt c = {0, 0};
    int n = p.size();
    for (int i = 0; i < n; i++) {
        ld crossProd = cross(p[i], p[(i + 1) % n]);
        A += crossProd;
        c += (p[i] + p[(i + 1) % n]) * crossProd;
    }
    A *= 0.5;
    c /= (6 * A);
    return c;
}

bool isConvex(const vector<pt> &p) {
    bool hasPos = false, hasNeg = false;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        T o = orient(p[i], p[(i+1)%n], p[(i+2)%n]);
        hasPos |= (o > 0);
        hasNeg |= (o < 0);
    }
    return !(hasPos && hasNeg);
}

// Returns 0, 1 or 2 centers of circles with radius r passing through a and b
int circle2ptRad(pt a, pt b, T r, pair<pt, pt>& out) {
    pt mid = (a + b) / 2;
    pt d = b - a;
    T d2 = sq(d);
    if (d2 == 0) return 0;
    T h2 = r*r - d2/4;
    if (h2 < 0) return 0;
    pt h = perp(d) * sqrt(h2 / d2);
    out = {mid - h, mid + h};
    return 1 + (h2 > 0);
}

```